```
# simple hash identifier


PREFIX_MAP = {
    "": "bcrypt",
    "": "bcrypt",
    "": "bcrypt",
    "$argon2": "Argon2"
}


LENGTH_MAP = {
    32: [("MD5", "MEDIUM"), ("NTLM", "LOW")],
    40: [("SHA1", "MEDIUM")],
    64: [("SHA256", "MEDIUM")]
}


def normalize_input(raw_hash):
    return raw_hash.strip()


def check_prefix(hash_str):
    for prefix, name in PREFIX_MAP.items():
        if hash_str.startswith(prefix):
            return [(name, "HIGH")]
    return []


def check_charset(hash_str):
    allowed = "0123456789abcdefABCDEF"
    for c in hash_str:
        if c not in allowed:
            return False
    return True


def check_length(hash_str):
    l = len(hash_str)
    if l in LENGTH_MAP:
        return LENGTH_MAP[l]
    return []


def identify_hash(raw_hash):
    clean = normalize_input(raw_hash)
    if clean == "":
        return []
```

```
    prefix_result = check_prefix(clean)
    if prefix_result != []:
        name = prefix_result[0][0]
        conf = prefix_result[0][1]
        return [{"type": name, "confidence": conf}]


    is_hex = check_charset(clean)
    if is_hex == False:
        return [{"type": "Unknown", "confidence": "LOW"}]


    length_result = check_length(clean)
    if length_result != []:
        final_list = []
        for name, conf in length_result:
            final_list.append({"type": name, "confidence": conf})
        return final_list


    return [{"type": "Unknown", "confidence": "LOW"}]


def format_results(results):
    if results == []:
        return "No hash found"
    text = ""
    for r in results:
        text = text + r["type"] + " [" + r["confidence"] + "], "
    return text


if __name__ == "__main__":
    user_input = input("Enter hash: ")
    result = identify_hash(user_input)
    print(format_results(result))
```